

Programación II

Desarrollo en Java

Clase N° 12:

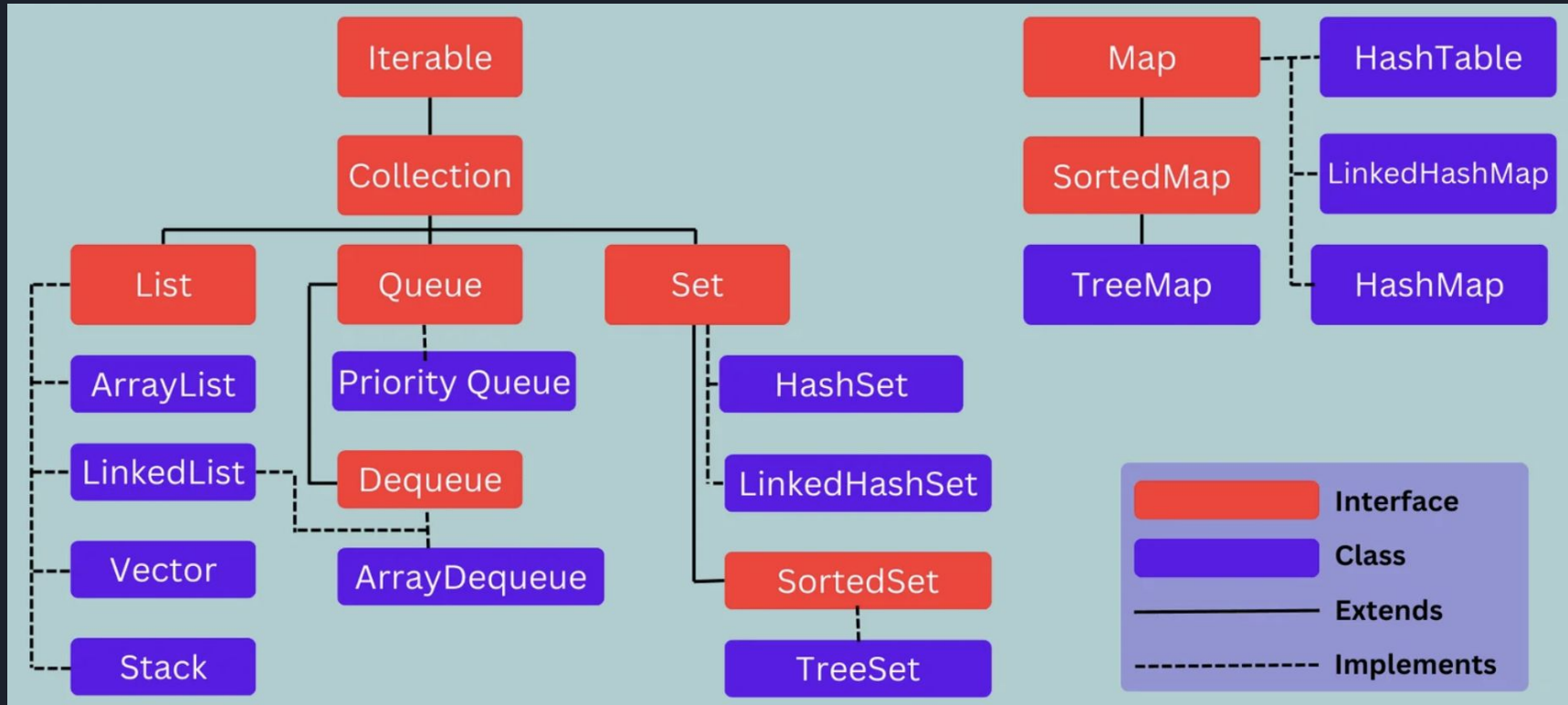
Colecciones.
List. Queue.

Docente Carolina Archuby.



API COLLECTION

La API Collection es un **conjunto de interfaces y clases** que se utilizan para **almacenar y manipular conjuntos de objetos** en forma de colecciones.



Ventajas de utilizar API Collection

La API Collection proporciona una serie de **MÉTODOS Y OPERACIONES COMUNES PARA MANIPULAR Y ACCEDER A LOS ELEMENTOS DE LAS COLECCIONES**, lo que permite manipularlas independientemente de los detalles de la implementación. **En consecuencia:**

=> **Reduce los esfuerzos de programación** al proveer estructuras de datos y algoritmos que no tenemos que escribir. Ofrece múltiples operaciones como búsqueda, inserción, eliminación y ordenación de elementos. (*¿Se acuerdan los dolores de cabeza con los algoritmos de ordenamiento de arreglos??... ¡con la API Collection se olvidan de eso!!*)

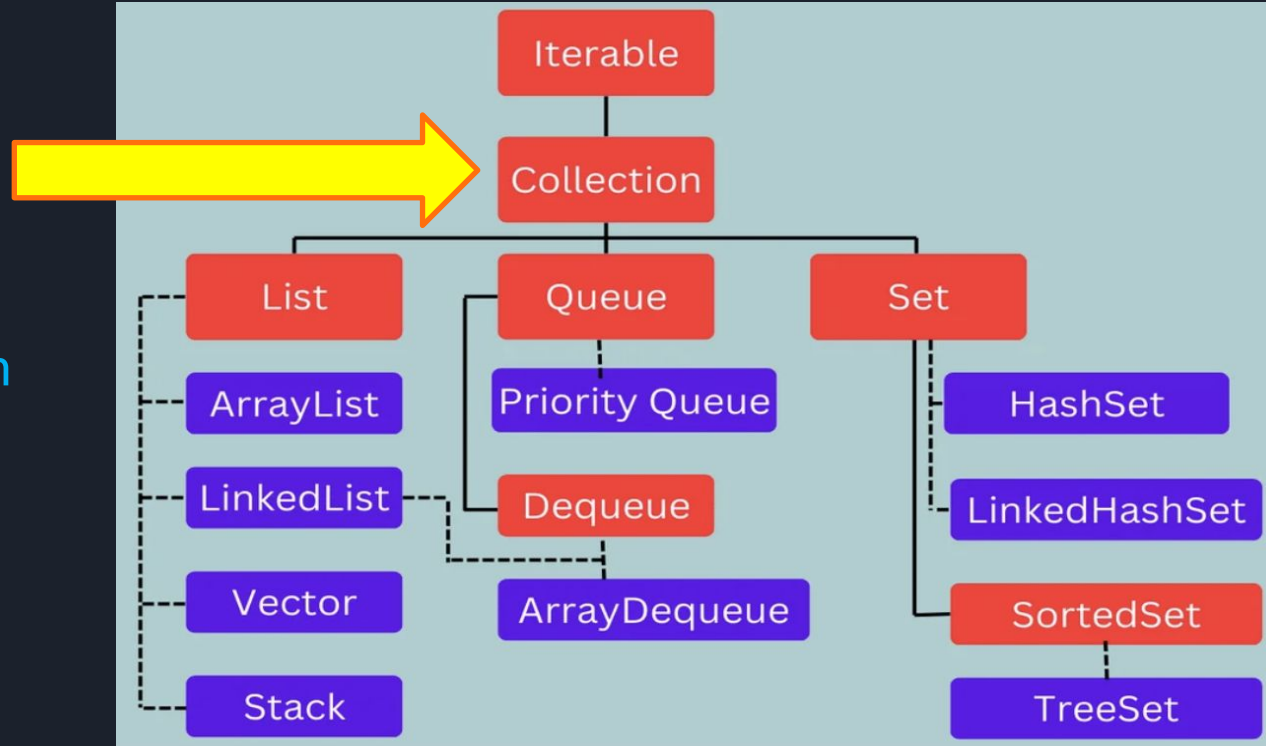
=> **Provee implementaciones de alta performance.**

=> **Fomenta la reutilización del software.**

Interfaz collection

Es la **RAÍZ DE TODAS LAS INTERFACES Y CLASES** relacionadas con colecciones de elementos SET, LIST y QUEUE:

Define operaciones básicas que obligatoriamente han de implementar las clases que implementan la interfaz Collection o cualquiera de las interfaces que hereden de Collection.



INTERFAZ COLLECTION

=> ¿Por qué se utiliza una interfaz? **Es la manera más genérica para representar un grupo de elementos**, permite una gran flexibilidad. Permite almacenar cualquier tipo de objeto y permite usar métodos comunes para manipular cualquier colección de elementos (por ejemplo: iterar, añadir, eliminar, obtener el tamaño de la colección, búsqueda, ordenamiento).

=> Al ser una interfaz y no una clase, obviamente **no se pueden construir objetos de tipo Collection**.

Interfaz Collection: Principales métodos DE INSTANCIA

(que van a implementar todas las colecciones List, Set y Queue):

- => **add(E e)**: Agrega un elemento a la colección.
- => **remove(Object o)**: Elimina de la lista la primera aparición del objeto especificado
- => **size()**: Retorna el número de elementos en la colección.
- => **isEmpty()**: Verifica si la colección está vacía.
- => **contains(Object o)**: Verifica si el objeto especificado está en la colección.
- => **iterator()**: Retorna un iterador sobre los elementos en la colección.
- => **clear()**: Elimina todos los elementos de la colección
- => **void addAll (Collection c)**
- => **Object[] toArray()**

Interfaz Collection: Principales métodos ESTÁTICOS:

=> boolean **addAll** (Collection c, T .. elements)

=> **Collections.addAll** (elementos, new CD(), new DVD());

=> void **copy** (List listaDestino, List listaOrigen)

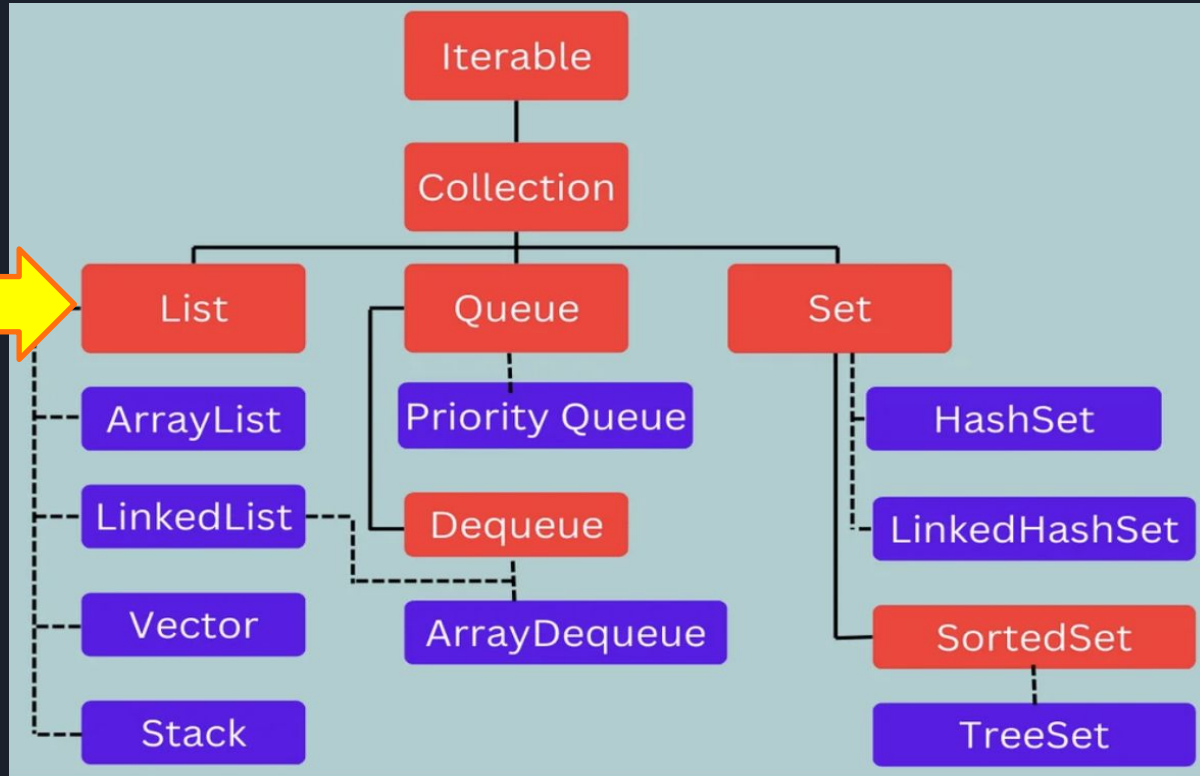
=> int **frequency** (Collection c, Object obj) (**obj debe implementar equals()*)

=> Object **max** (Collection c) (**los elementos tienen que implementar Comparable*)

=> Object **min** (Collection c) (**los elementos tienen que implementar Comparable*)

=> void **reverse** (List lista)

INTERFACE LIST



INTERFACE LIST

Características distintivas:

- => Permite agrupar una **colección de elementos en forma ordenada o SECUENCIAL** (uno detrás del otro)
- => Permite el **ACCESO A SUS ELEMENTOS MEDIANTE UN ÍNDICE** que representa su ubicación numérica dentro de la lista.
- => **Permite elementos DUPLICADOS** en la lista.

INTERFACE LIST

Características distintivas:

- => Permite agrupar una **colección de elementos en forma ordenada o SECUENCIAL** (uno detrás del otro)
- => Permite el **ACCESO A SUS ELEMENTOS MEDIANTE UN ÍNDICE** que representa su ubicación numérica dentro de la lista.
- => **Permite elementos DUPLICADOS** en la lista.

Métodos más importantes de la Interface List

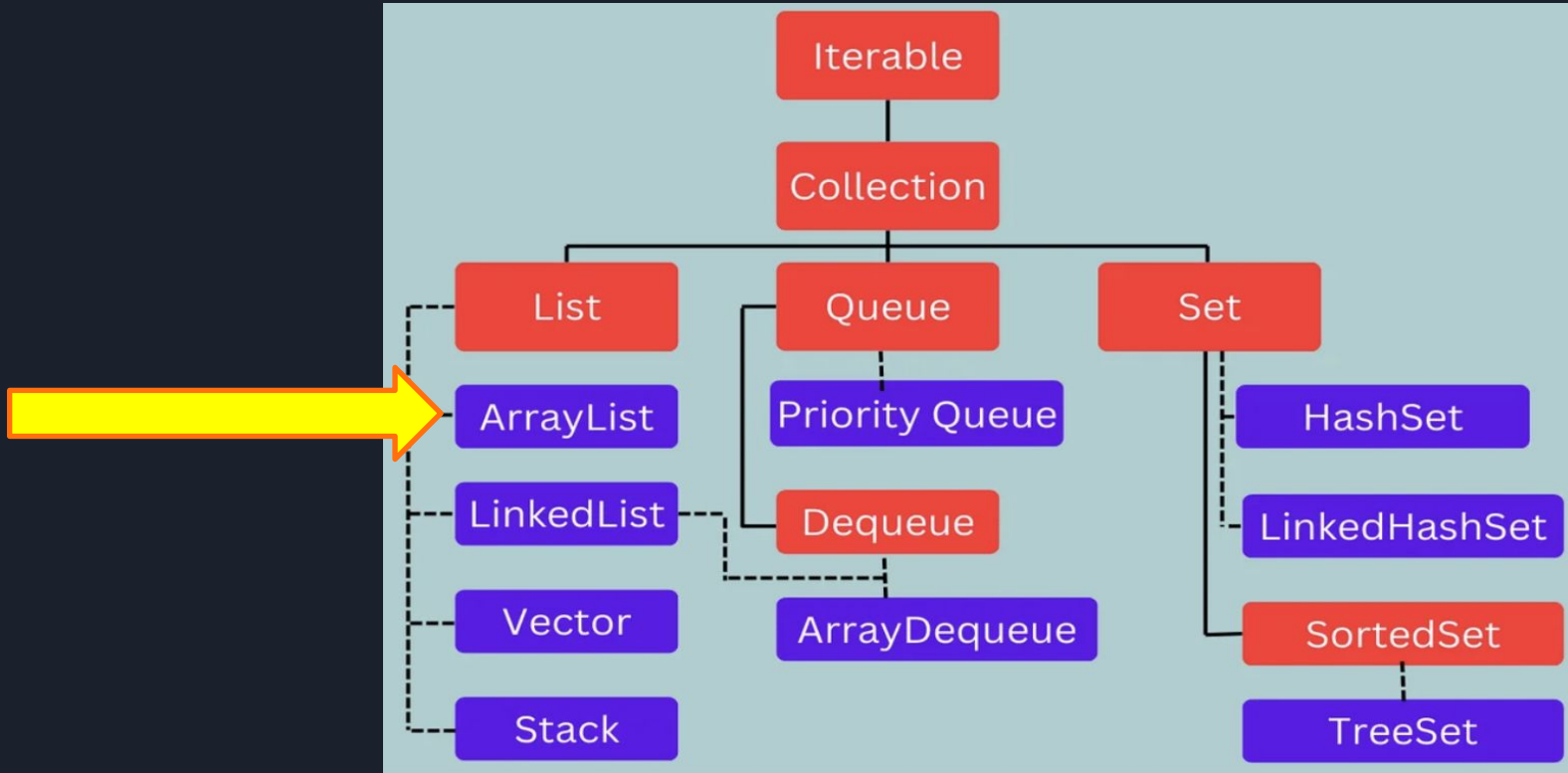
Además de los que hereda de la interfaz Collection (add, addAll, remove, clear, contains, size, isEmpty, etc), agrega **métodos relacionados con el acceso posicional a los elementos**:

- **Add** (int índice, E elemento): Inserta un elemento en la posición especificada por el índice, desplazando los elementos existentes hacia adelante.
- **Object remove** (int índice): Elimina el elemento en la posición especificada por el índice, desplazando hacia atrás los elementos subsiguientes.
- **Object Get** (int índice): Devuelve el elemento en la posición especificada por el índice.

Métodos más importantes de la Interface List

- **Object set** (int index, Object element): Reemplaza el elemento en la posición index con element.
- **int indexOf** (Object objeto): Devuelve el índice de la primera aparición del objeto especificado en la lista, o -1 si no está en la lista.
- **int lastIndexOf** (Object objeto): Devuelve el índice de la última aparición del objeto especificado en la lista, o -1 si no está en la lista.
- **subList** (int inicio, int fin): Devuelve una vista de sublista de la lista original, que abarca desde el índice de inicio (inclusive) hasta el índice de fin (exclusive).
- **listIterator**(): Devuelve un ListIterator que permite la iteración bidireccional (hacia adelante y hacia atrás) y permite añadir, modificar y eliminar elementos mientras se itera.

ARRAYLIST



ARRAYLIST

Es una CLASE que permite almacenar datos de forma **similar a un ARRAY** pero de forma DINÁMICA, con lo cual hay dos ventajas:

=> **NO ES NECESARIO DECLARAR SU TAMAÑO INICIAL**, como pasa con los Arrays.

=> **PUEDE IR CRECIENDO SI ES NECESARIO**. Cuando se añaden elementos y se supera la capacidad, ArrayList automáticamente crea un nuevo array más grande, copia los elementos del array antiguo al nuevo, y descarta el antiguo (proceso llamado **redimensionamiento**)

ARRAYLIST: Ventajas

=> Las que ya posee por el hecho de implementar la Interfaz List-> acceso directo a los elementos mediante índice-> **las operaciones de acceso a elementos, capacidad y saber si está vacío se realizan de forma eficiente y rápida**

=> Más los beneficios relacionados con que sea un arreglo **DINÁMICO**-> **No necesita definir dimensión inicial y redimensionamiento.**

ARRAYLIST: Desventajas

- => **Añadir elementos puede ser lento si se requiere redimensionar el array**, especialmente si el array es grande, porque implica copiar todos los elementos al nuevo array.
- => **Añadir y eliminar elementos en el medio de la lista es lento** porque implica mover todos los elementos subsiguientes para mantener el orden.
- => **No están sincronizados**. Con lo cual, si múltiples hilos acceden a un mismo ArrayList concurrentemente, podríamos tener problemas en la consistencia de los datos. Por lo tanto, al usar ArrayList habrá que controlar la concurrencia de acceso.

ARRAYLIST: Distintas formas de declaración

//En las viejas versiones de JAVA era necesario poner el tipo de dato en ambos lados

```
ArrayList<String> listaA = new ArrayList<String>();
```

// Hoy no es necesario poner el tipo de lado en ambos lados

```
ArrayList<String> listaB = new ArrayList<>();
```

// Puede usarse la interfaz como tipo de dato

```
List<String> listaC = new ArrayList<String>();
```

// Puede darse o no una dimension inicial al ArrayList (si no, por defecto es 10)

```
ArrayList<String> listaD = new ArrayList<>(initialCapacity: 20);
```

ArrayList – Inserir elemento.

```
public static void main(String [] args) {
```

```
List<Perro> perros = new ArrayList<>();
```

Construye un arreglo interno con una capacidad de 10.

```
Perro p1 = new Perro("Boby");
```

```
Perro perro2 = new Perro("Ayudante de Santa");
```

```
perros.add(perro1);
```

```
perros.add(perro2);
```

Antes de agregar un elemento, se valida la capacidad del arreglo.

}

[illegible]

ArrayList – Insertar elemento con redimensionamiento

```
public static void main(String [] args) {
```

```
    List<Perro> perros = new ArrayList<>();
```

```
    Perro p1 = new Perro("Boby");
```

```
    Perro p2 = new Perro("Ayudante de Santa");
```

```
    ...
```

```
    Perro p11 = new Perro("Rob")
```

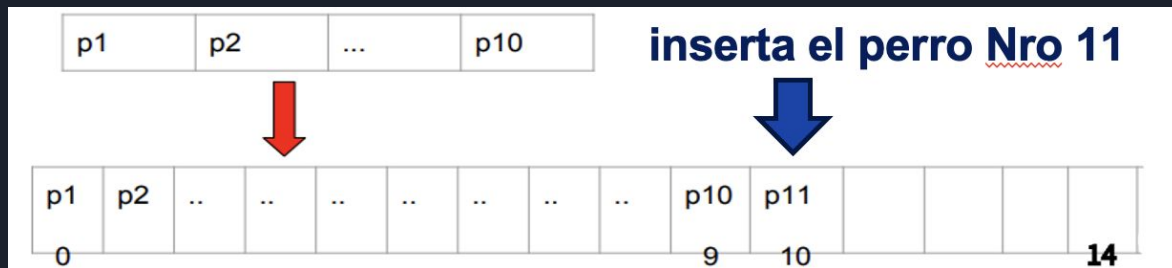
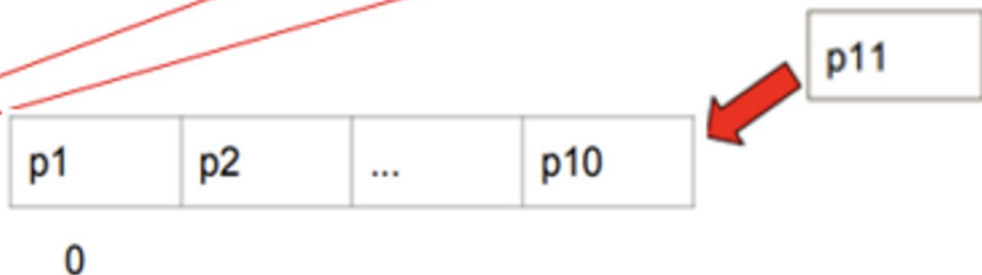
```
    perros.add(p1);
```

```
    ...
```

```
    perros.add(p11);
```

```
}
```

Se valida la capacidad del arreglo y como no es suficiente se crea un nuevo arreglo con un 50% más de la capacidad y se copia el anterior.



ArrayList – Insertar elemento por índice:

Ejemplo: Agregar un nuevo elemento en la posición 1:

```
nombreArrayList.add ( index: 1, element: "Elemento");
```

ArrayList – Modificar un elemento por índice:

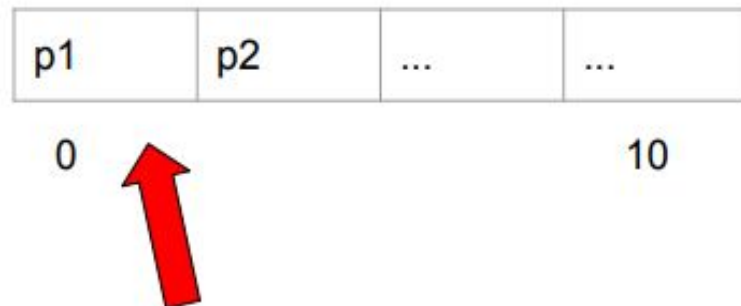
Ejemplo: Reemplazar el elemento de la posición 1 por uno nuevo:

```
nombreArrayList.set(1, "nuevoElemento");
```

ArrayList – Lectura de elementos

- Como la estructura interna es un arreglo, se accede al elemento a través del índice correspondiente.

```
public static void main(String [] args) {  
  
    List<Perro> perros = new ArrayList<>();  
  
    Perro p1 = new Perro("Boby");  
    Perro p2 = new Perro("Ayudante de Santa");  
  
    perros.add(p1);  
  
    System.out.println(perros.get(0).getNombre());  
}
```



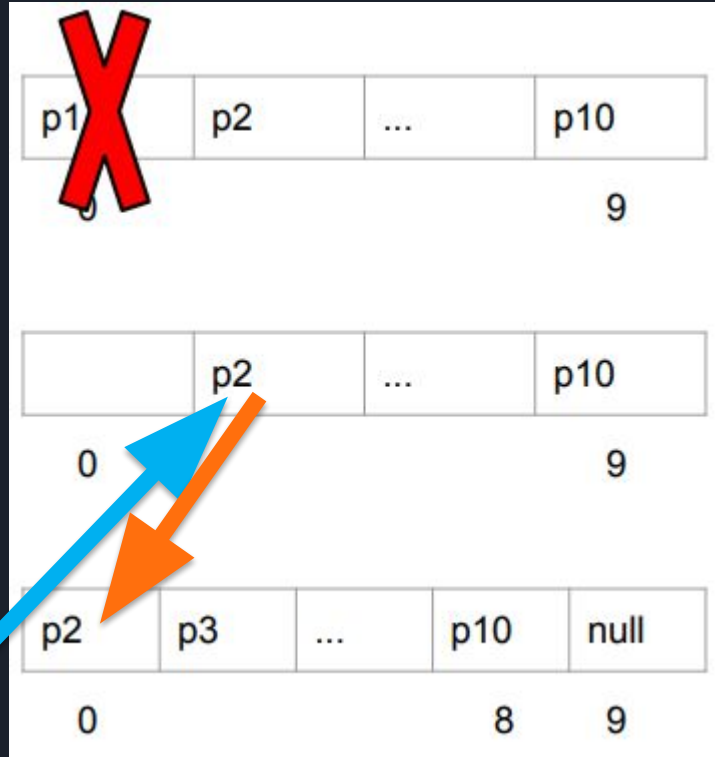
Es lo mismo que: `p1.getNombre()`

ArrayList – Eliminar elementos a través del índice

Borra el elemento de la posición elegida del ArrayList

```
public static void main(String [] args) {  
  
    List<Perro> perros = new ArrayList<>();  
    Perro p1 = new Perro("Boby");  
    perros.add(p1);  
    perros.remove(0);  
}
```

Se deben correr los elementos



ArrayList – Eliminar elementos a través de la igualdad

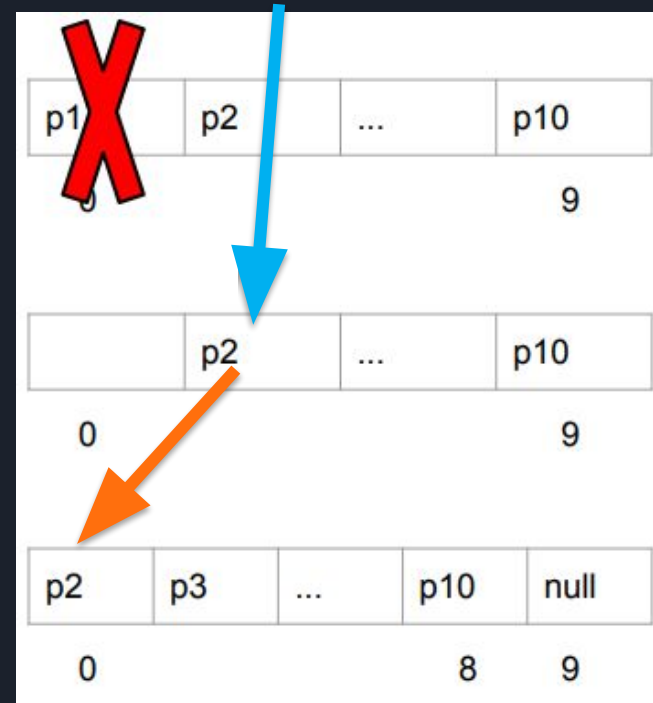
Borra la primera ocurrencia del elemento que se pasa como parámetro

```
public static void main(String [] args) {  
    List<Perro> perros = new ArrayList<>();  
    Perro p1 = new Perro("Boby");  
    perros.add(p1);  
    perros.remove(p1);  
}
```

También se deben correr los elementos

Se evalúa cada elemento del ArrayList para ver si es igual al que se quiere eliminar:
if (elementoArray.equals(p1))

¡OJO!! DEBE SOBRESERIBIRSE METODO equals() EN LAS CLASES PROPIAS



Algunos ejemplos de métodos que vienen de la interfaz List y de la interfaz Collection

```
// Búsqueda  
int primeraPosicionJuan = nombres.indexOf("Juan"); // Devuelve 3  
int ultimaPosicionJuan = nombres.lastIndexOf(o: "Juan"); // Devuelve 3  
  
// Rango de operación  
List<String> subNombres = nombres.subList(1, 3); // Obtiene [Carlos, Pedro]
```


Iterar un ArrayList

A) Con un for o while

```
for(int i=0; i<arrayName.size(); i++) {  
  
    System.out.println(al.get(i));  
  
}
```

B) Con un for each:

```
for (String nombre : lista) {  
  
    System.out.println(nombre);  
  
}
```

C) Con un iterador

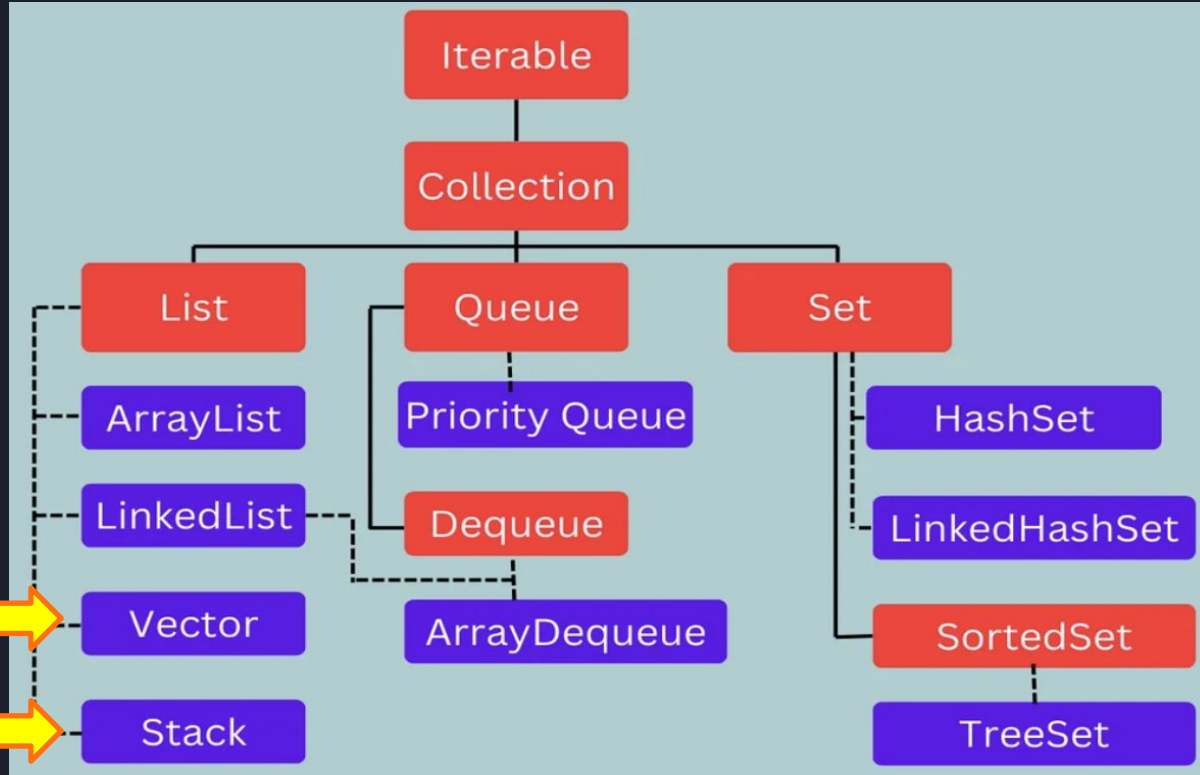
```
ListIterator<String> iterador = nombres.listIterator();  
while (iterador.hasNext()) {  
    System.out.println(iterador.next());  
    // Puedes también moverte hacia atrás con iterador.previous()  
}
```

Recorrer un ArrayList ... OJO!!

=> **CON UN FOR EACH NO SE PUEDEN AGREGAR O ELIMINAR DATOS DE UN LISTADO MIENTRAS SE LO RECORRE** (se puede modificar internamente un ELEMENTO, pero NO EL LISTADO MISMO).

=> **Sí puede modificarse el listado usando for o while con índice, o Iterator.**

VECTOR y STACK



VECTOR y STACK

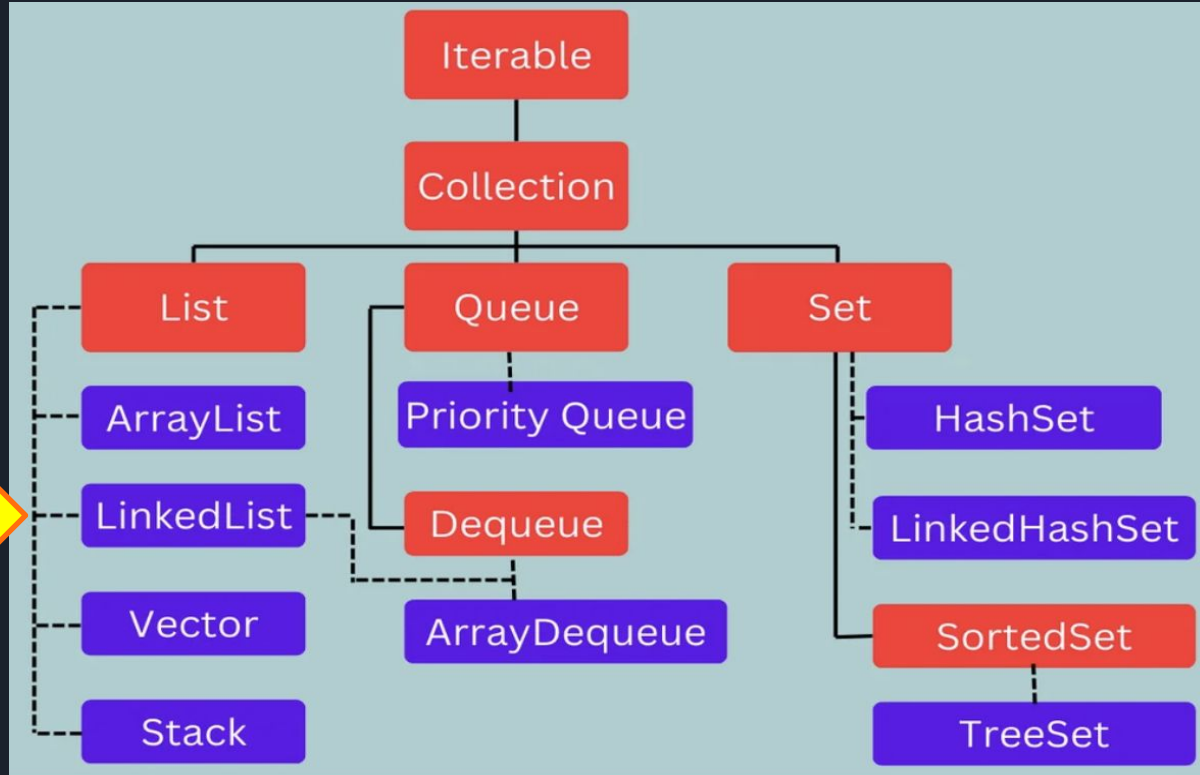
=> Un **VECTOR** es similar a un ArrayList (es un **arreglo dinámico**, usa índice, y permite elementos repetidos), **pero la diferencia es que está SINCRONIZADO**.

⇒ **En la actualidad no se usa.**

=> Una **STACK** es una **SUBCLASE DE VECTOR**, por lo tanto, también es **SINCRONIZADA**, y por lo tanto, su estructura también está basada en un **array dinámico**.

La diferencia es que representa una estructura **LIFO** (last in - first out).

LINKEDLIST



LINKEDLIST

=> Su implementación se basa en una **LISTA DOBLEMENTE ENLAZADA DE TAMAÑO ILIMITADO**.

=> También implementa la interfaz List, por lo tanto **puede accederse a los elementos por posición**.

=> Su estructura está formada por **Nodos**. Cada nodo contiene tres componentes: el dato, un enlace a su nodo anterior y otro enlace a su nodo siguiente. Esto permite recorrer la lista en ambos sentidos.

LinkedList vs ArrayList

ArrayList está basada en una estructura de datos del tipo arreglo, mientras que LinkedList está basada en una lista doblemente vinculada.

0	1	2	3	4
23	3	17	9	42



LinkedList vs ArrayList

=> Almacenar elementos en un **ArrayList** consume **menos memoria**, ya que la **LinkedList** ocupa **más memoria por cada elemento**, porque debe almacenar y gestionar los enlaces al nodo anterior y al siguiente.

=> El **acceso a los elementos por índice** es más rápido en el **ArrayList**, porque es directo, en cambio **en la LinkedList hay que recorrer toda la lista hasta encontrar el elemento deseado**.

=> **Agregar o eliminar elementos** usualmente es más rápido en **LinkedList**, por eso es **ideal para colecciones donde las inserciones y eliminaciones son frecuentes**.

LinkedList vs ArrayList

Sin embargo, todo depende de la posición del elemento a agregar o eliminar:

- * Si agregamos o eliminamos un elemento al final, será más rápido ArrayList que LinkedList, porque accedemos directamente a la posición y no hay que modificar enlaces entre nodos.
- * Pero si necesitamos agregar o eliminar un dato en otro lugar de la lista, en el ArrayList habrá que mover todo el resto de los datos, por lo cual será más rápida la LinkedList, en la cual para agregar o eliminar un elemento sólo habrá que cambiar los enlaces de los nodos.
- * Sin embargo, como en la LinkedList se debe iterar hasta la posición en la que se desea agregar o eliminar el elemento, la pérdida de rendimiento a veces es más grande que la ganancia.

LinkedList vs ArrayList

RESUMEN

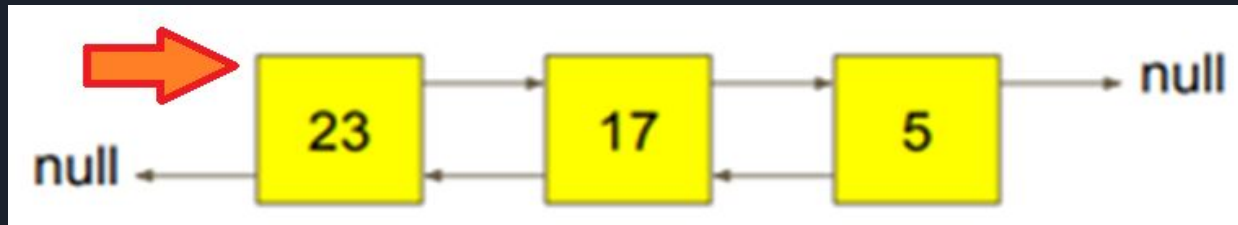
=> el **ArrayList** es mas conveniente en aplicaciones que requieren búsqueda frecuente de elementos por su índice y en las cuales las adiciones y eliminaciones se realizan principalmente al final de la lista.

=> y la **LinkedList** es mas conveniente en aplicaciones que requieren inserciones y eliminaciones frecuentes.

LinkedList: Agregar elemento

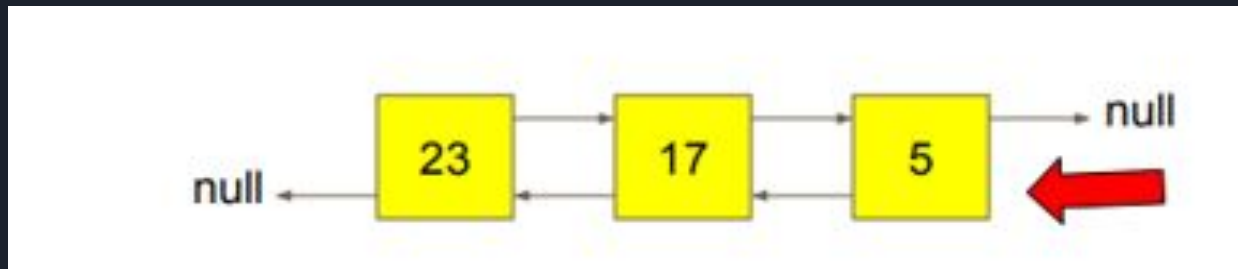
Agregar al principio

void **addFirst** (Object obj)
(se accede directamente al inicio de la lista)



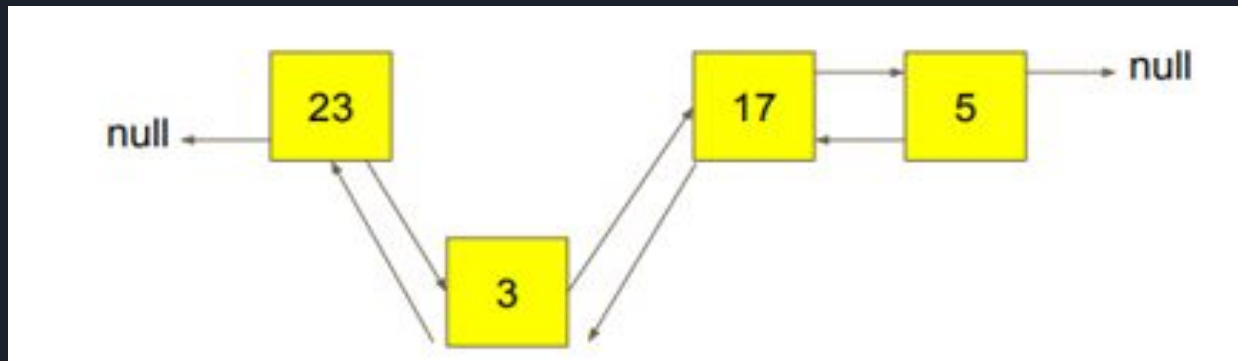
Agregar al final

void **addLast** (Object obj)
(se accede directamente al final de la lista)



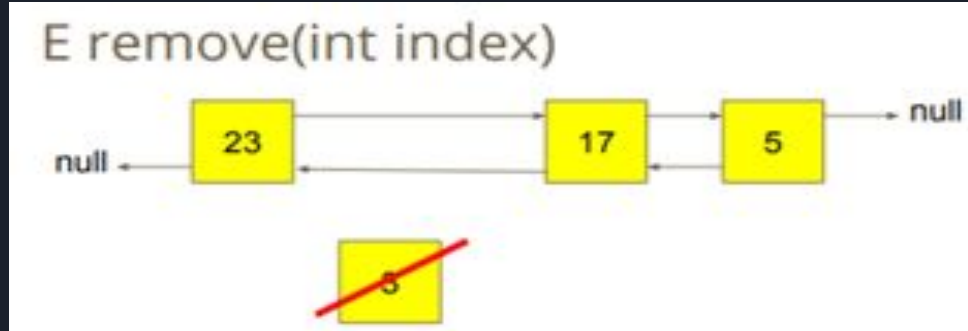
Agregar por índice

void **add** (int index, Object o)
(menos eficiente)

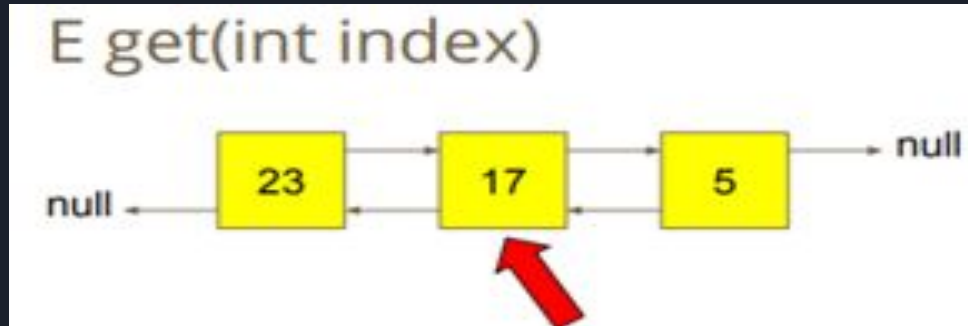


LinkedList: Eliminar y leer elementos

Eliminar elemento
por índice



Leer elemento
por índice



OTRAS FORMAS:

=> Object **getFirst()** / Object **getLast()**

=> Object **removeFirst()** / Object **removeLast()**

LinkedList: Ejemplo de código

```
// Creación de un LinkedList
    LinkedList<String> tareas = new LinkedList<>();
// Añadir elementos al final
    tareas.add("Lavar la ropa");
    tareas.add("Pagar cuentas");
// Añadir al principio
    tareas.addFirst(e: "Sacar al perro");
// Añadir al final
    tareas.addLast(e: "Hacer la compra") ;
// Eliminar el primer elemento
    tareas.removeFirst();
// Eliminar el último elemento
    tareas.removeLast();
// Imprimir la lista
    for (String tarea : tareas) {
        System.out.println(tarea);
    }
```

LA INTERFAZ COMPARABLE

=> La interfaz Comparable en Java es una forma de definir el orden natural de los objetos de una clase. **Al implementar Comparable, una clase indica que sus instancias se pueden ordenar de acuerdo a un orden específico,** generalmente en una secuencia ascendente o descendente.

=> La interfaz Comparable es fundamental para varias estructuras de datos de Java que dependen de un orden, como TreeSet y TreeMap, y para algoritmos de ordenación como Arrays.sort() y Collections.sort().

INTERFAZ COMPARABLE: Método compareTo

El método compareTo(Object o) **ES EL ÚNICO MÉTODO DE LA INTERFAZ COMPARABLE**. Un objeto a se compara con otro objeto b mediante a.compareTo(b).

```
public interface Comparable<T> {  
    int compareTo(T t);  
}
```

CONTRATO:

El contrato de este método especifica que debe devolver lo siguiente:

- **Entero negativo** si el objeto es menor al especificado por parámetro.
- **Cero** si el objeto es igual al especificado por parámetro.
- **Entero positivo** si el objeto es mayor al especificado por parámetro.

INTERFAZ COMPARABLE:

Consideraciones de implementación

ES IMPORTANTE QUE `compareTo` TO SEA CONSISTENTE CON `EQUALS` => si `a.compareTo(b) == 0`,

Entonces: `a.equals(b)` debería retornar `true`.

Esto no es obligatorio, pero muchas clases y métodos en Java asumen esta consistencia

Interfaz comparable: ejemplo 1 (comparaciones múltiples anidadas)

```
public class Telephone {
    private short areaCode;
    private short prefix;
    private short lineNum;
    ...
    public int compareTo(Telephone tp) {
        int result = Short.compare(areaCode, tp.areaCode);
        if (result == 0) {
            result = Short.compare(prefix, tp.prefix);
            if (result == 0) result = Short.compare(lineNum, tp.lineNum);
        }
        return result;
    }
}
```

Interfaz comparable:

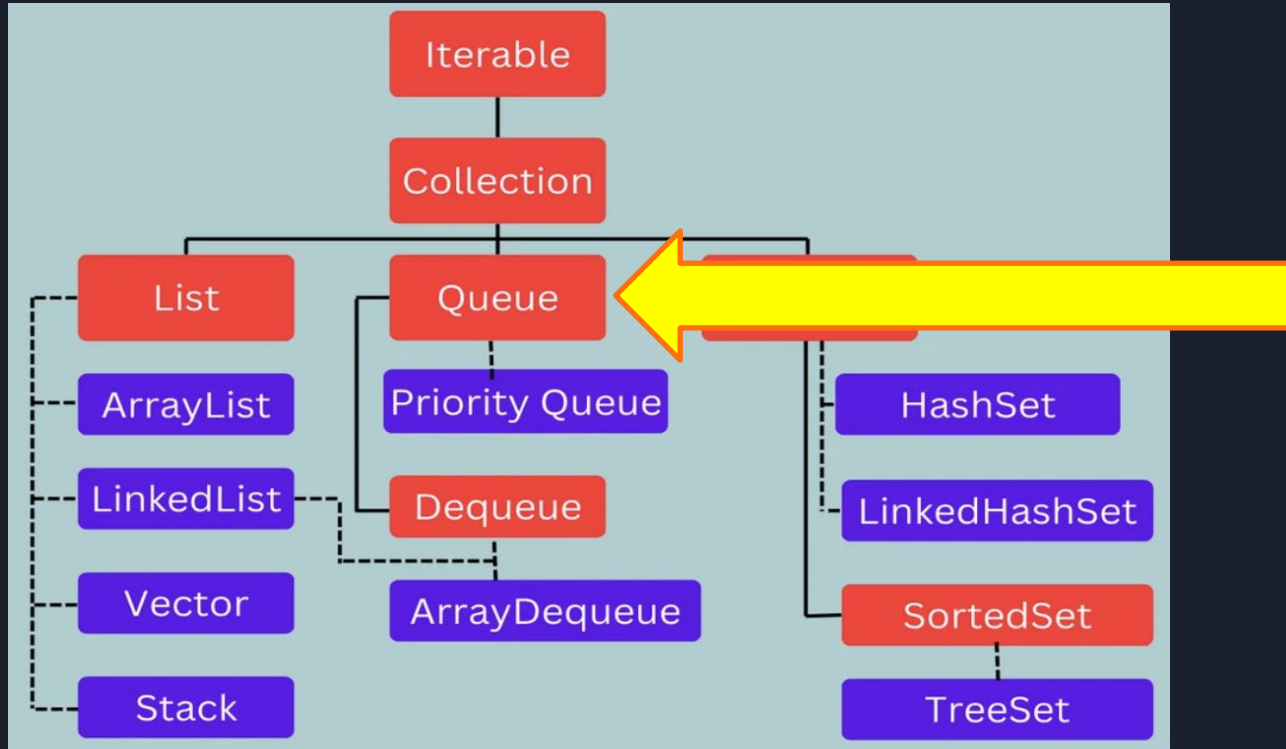
ejemplo 2:

```
public class Persona implements Comparable<Persona> {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    // setters y getters  
  
    @Override  
    public int compareTo(Persona otra) {  
        return Integer.compare(this.edad, otra.edad);  
    }  
  
    @Override  
    public String toString() {  
        return nombre + " (" + edad + ")";  
    }  
}
```

Y ahora con
ello podemos
ordenar una
lista de
personas:

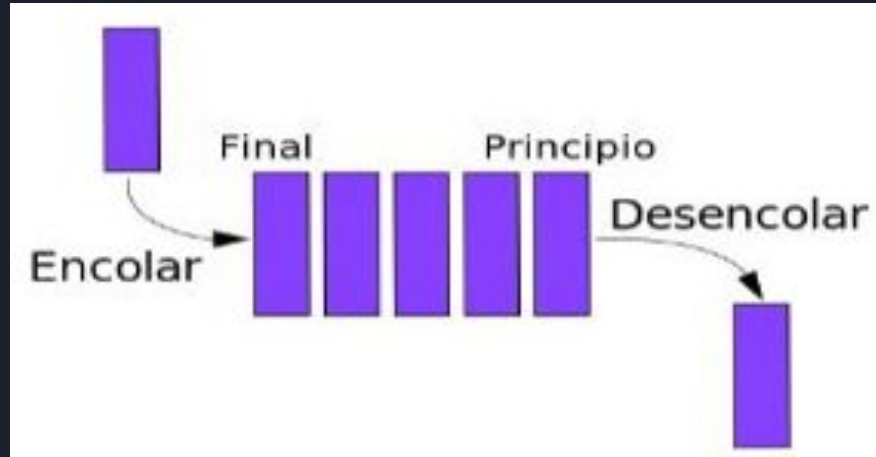
```
List<Persona> personas = new ArrayList<>();  
personas.add(new Persona("Juan", 25));  
personas.add(new Persona("Ana", 23));  
personas.add(new Persona("Luis", 30));  
  
Collections.sort(personas);  
  
for (Persona p : personas) {  
    System.out.println(p);  
}  
  
// Imprime:  
// Ana (23)  
// Juan (25)  
// Luis (30)
```

INTERFAZ QUEUE



INTERFAZ QUEUE

Posee métodos para representar el comportamiento **FIFO** (First in – First out), que representa al tipo de dato **COLA** o **FILA**, que es una lista en la que **sus elementos se introducen** únicamente por un extremo (**fin** de la cola) **y se remueven por** el extremo contrario (**principio** de la cola).

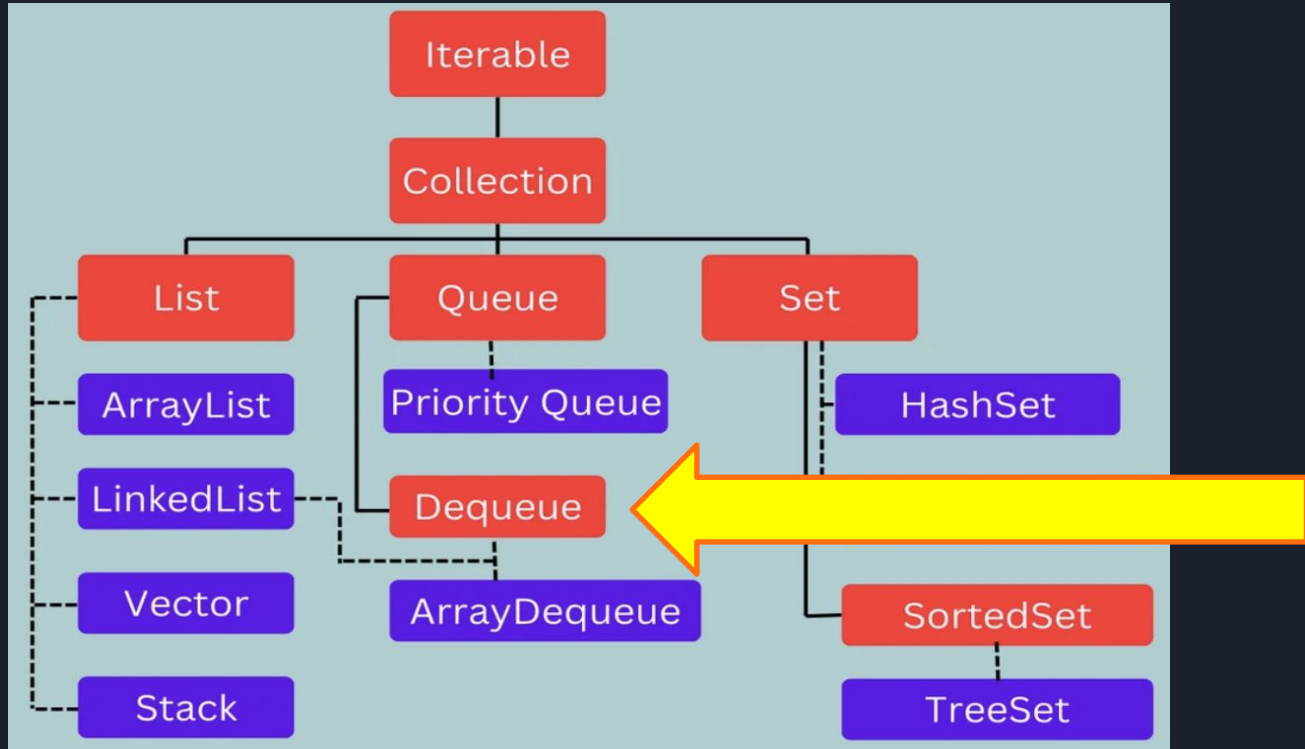


INTERFAZ QUEUE. Métodos:

Posee los métodos heredados de la interfaz Collection (add, remove, etc) más métodos específicos para manejar el comportamiento FIFO:

- **offer (E e):** Inserta el elemento al **final** de la cola si es posible hacerlo o retorna false si no hay espacio.
- **E poll ():** Elimina y retorna el **primer** elemento de la cola, o devuelve null si la cola está vacía.
- **E peek ():** Retorna, pero no elimina, el **primer** elemento de la cola, o devuelve null si la cola está vacía.

INTERFAZ DEQUEUE



INTERFACE DEQUEUE

Hereda de Queue y agrega métodos para representar el comportamiento **LIFO** (Last in – First out), que representa al tipo de dato **PILA**, que es una lista en la que **sus elementos se introducen y se remueven por el tope**

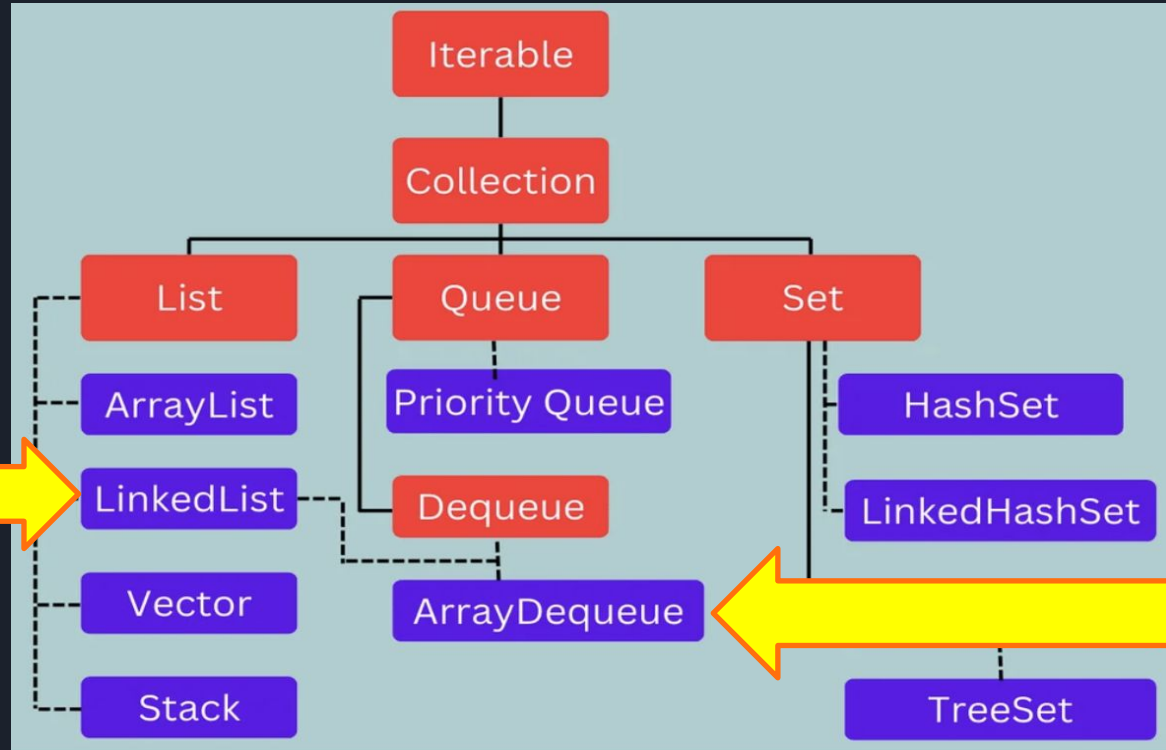


INTERFAZ DEQUEUE. Métodos:

Posee los métodos heredados de las interfaces Collection y Queue más
métodos específicos para manejar el comportamiento LIFO:

- => **push** → introduce un elemento en el tope de la pila.
- => **pop** → quita y retorna el elemento del tope de la pila.
- => **peek** → retorna, sin eliminar, el primer elemento del tope de la pila.
- => **empty** → comprueba si la pila está vacía.

CLASES QUE IMPLEMENTAN DEQUEUE:



ArrayDeque y LinkedList

=> **ArrayDeque y LinkedList** implementan la interfaz Dequeue (y por ende, también su interfaz padre Queue), por lo tanto **implementan tanto los métodos para funcionar como una PILA como los métodos para funcionar como una FILA**. El comportamiento, entonces, dependerá de los métodos usados:

- * Para **pilas LIFO** → **push, pop, peek**.
- * Para **colas FIFO** → **offer, poll, peek**.

=> La diferencia entre ambas clases es que **LinkedList** además implementa la interfaz **List**, por lo tanto se maneja con índice, en cambio **ArrayDeque** no tiene acceso por índice.